

## ASSIGNMENT -1.5

NAME : JAWWAD KHAN

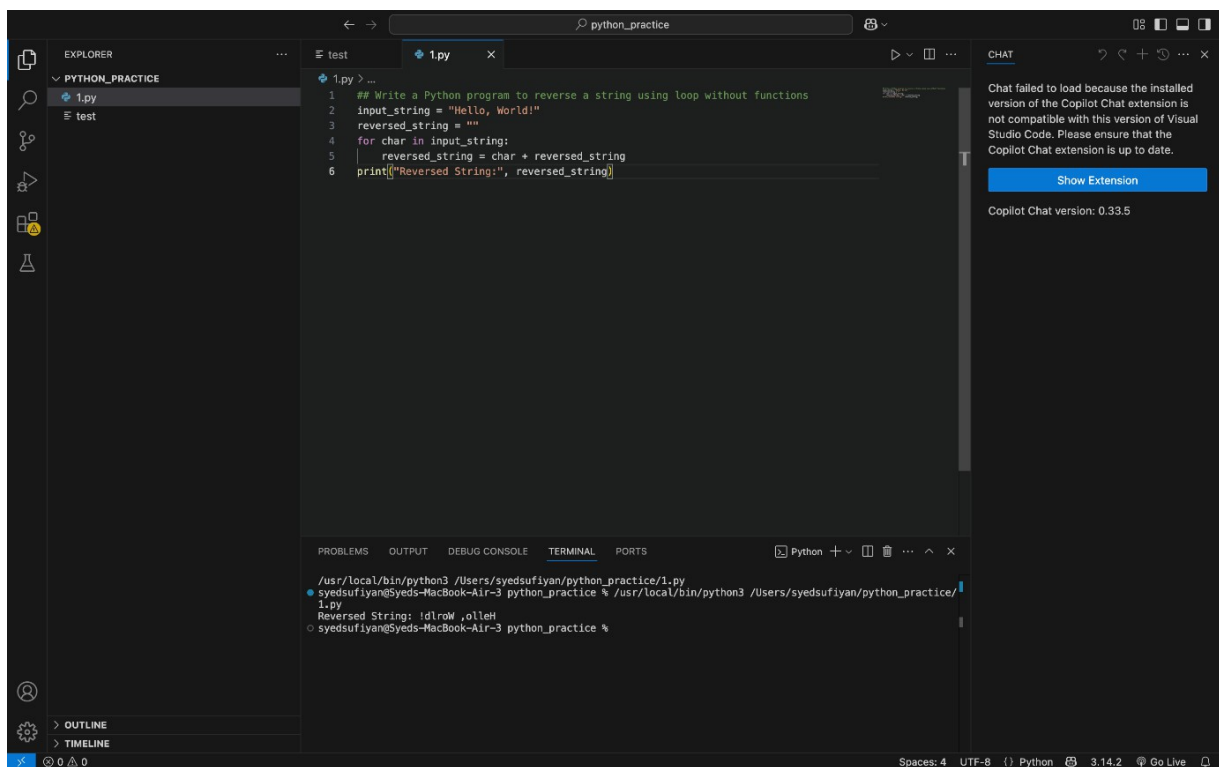
2303A51059

BATCH:29

### TASK 1:

PROMPT: AI-GENERATED LOGIC WITHOUT MODULARIZATION  
(STRING REVERSAL WITHOUT FUNCTIONS)

CODE:



The screenshot displays the Visual Studio Code interface. The Explorer panel on the left shows a project named 'PYTHON\_PRACTICE' with files '1.py' and 'test'. The main editor window shows the content of '1.py', which contains a Python script to reverse a string using a loop. The script is as follows:

```
1 ## Write a Python program to reverse a string using loop without functions
2 input_string = "Hello, World!"
3 reversed_string = ""
4 for char in input_string:
5     reversed_string = char + reversed_string
6 print("Reversed String:", reversed_string)
```

The bottom panel shows the TERMINAL output, indicating the script was executed successfully and produced the reversed string:

```
/usr/local/bin/python3 /Users/syedsufiyan/python_practice/1.py
syedsufiyan@Syeds-MacBook-Air-3 python_practice % /usr/local/bin/python3 /Users/syedsufiyan/python_practice/1.py
Reversed String: !dlroW ,olleH
syedsufiyan@Syeds-MacBook-Air-3 python_practice %
```

On the right side, the CHAT panel shows a message: 'Chat failed to load because the installed version of the Copilot Chat extension is not compatible with this version of Visual Studio Code. Please ensure that the Copilot Chat extension is up to date.' Below this message is a 'Show Extension' button and the text 'Copilot Chat version: 0.33.5'.

### OBSERVATION:

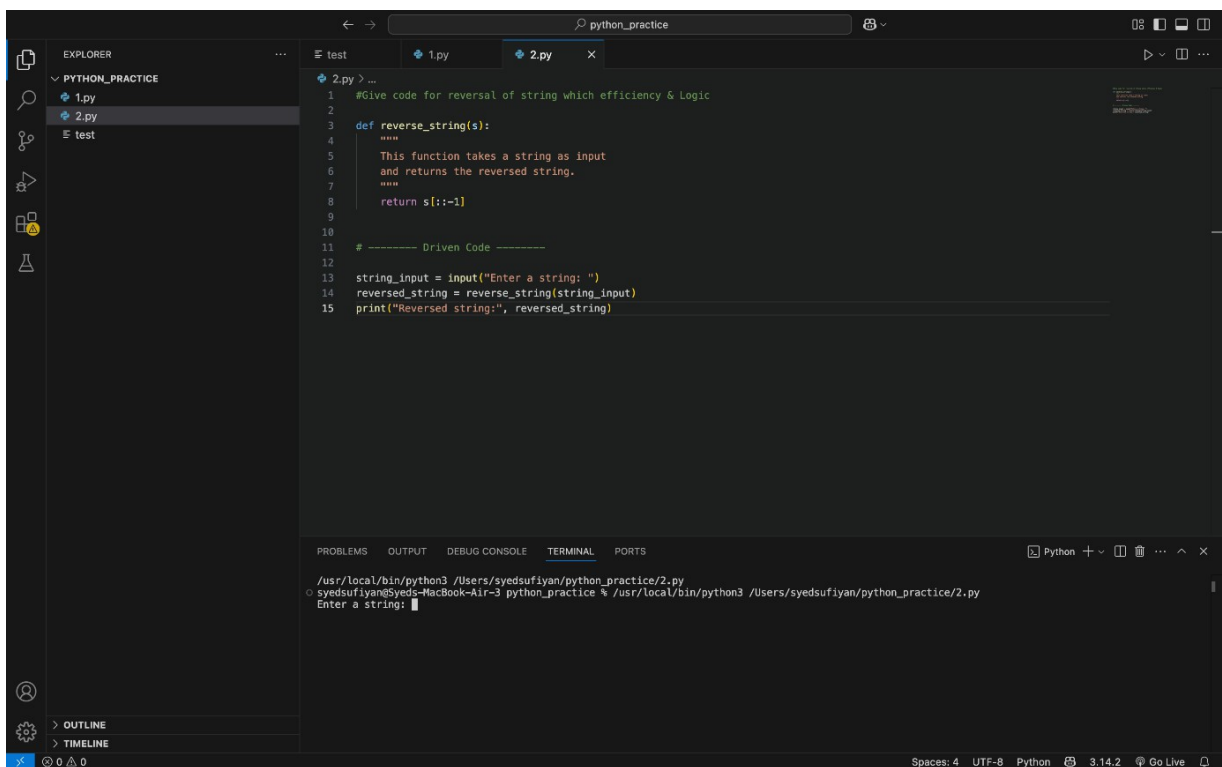
The program successfully reverses the given string using a simple loop without using any functions. Each character is added to the beginning of a new string, which gradually forms the reversed output. The output confirms that the logic works correctly by displaying the reversed string. This approach is easy to understand

and suitable for beginners learning basic string operations. However, for larger programs, a more optimized or modular approach would be better.

## TASK:2

PROMPT: GIVE CODE FOR REVERSAL OF STRING WHICH EFFICIENCY & LOGIC OPTIMIZATION

CODE:



```
1 #Give code for reversal of string which efficiency & Logic
2
3 def reverse_string(s):
4     """
5     This function takes a string as input
6     and returns the reversed string.
7     """
8     return s[::-1]
9
10
11 # ----- Driven Code -----
12
13 string_input = input("Enter a string: ")
14 reversed_string = reverse_string(string_input)
15 print("Reversed string:", reversed_string)
```

The screenshot shows a VS Code editor with a file named '2.py' open. The code defines a function 'reverse\_string(s)' that uses slicing 's[::-1]' to reverse a string. Below the function, there is a 'Driven Code' section that takes user input, calls the function, and prints the result. The terminal at the bottom shows the command to run the script and the prompt 'Enter a string: '.

OBSERVATION:

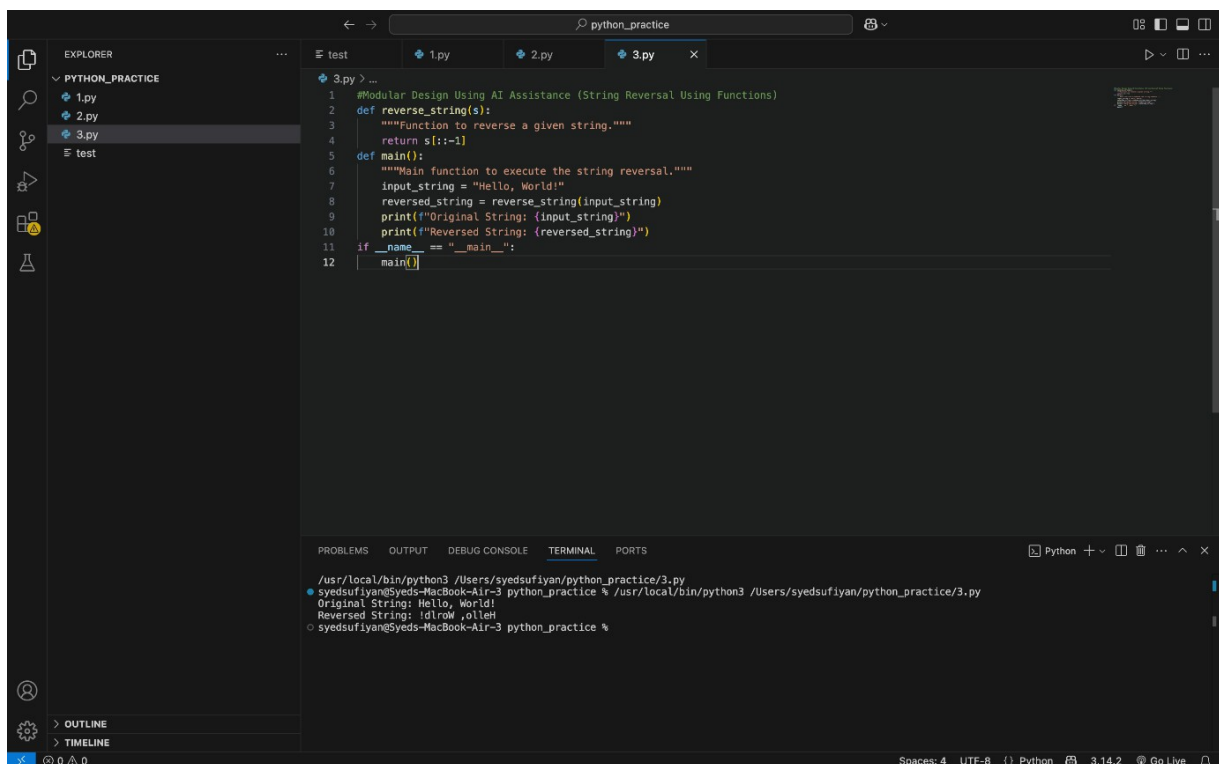
The function uses Python slicing to reverse the string in a single step. No extra variables or loops are used, which makes the code easy to read. The logic is efficient and executes faster than manual reversal methods. Overall, the code is clean, readable, and suitable for review by other developers. The optimized approach reduces unnecessary

operations and improves performance. It follows Python best practices, making the code more maintainable and reliable.

### TASK:3

PROMPT: MODULAR DESIGN USING AI ASSISTANCE (STRING REVERSAL USING FUNCTIONS)

CODE:



```
1 #Modular Design Using AI Assistance (String Reversal Using Functions)
2 def reverse_string(s):
3     """Function to reverse a given string."""
4     return s[::-1]
5
6 def main():
7     """Main function to execute the string reversal."""
8     input_string = "Hello, World!"
9     reversed_string = reverse_string(input_string)
10    print(f"Original String: {input_string}")
11    print(f"Reversed String: {reversed_string}")
12
13 if __name__ == "__main__":
14     main()
```

```
/usr/local/bin/python3 /Users/syedsufiyan/python_practice/3.py
Original String: Hello, World!
Reversed String: !dlroW ,olleH
syedsufiyan@Syeds-MacBook-Air-3 python_practice %
```

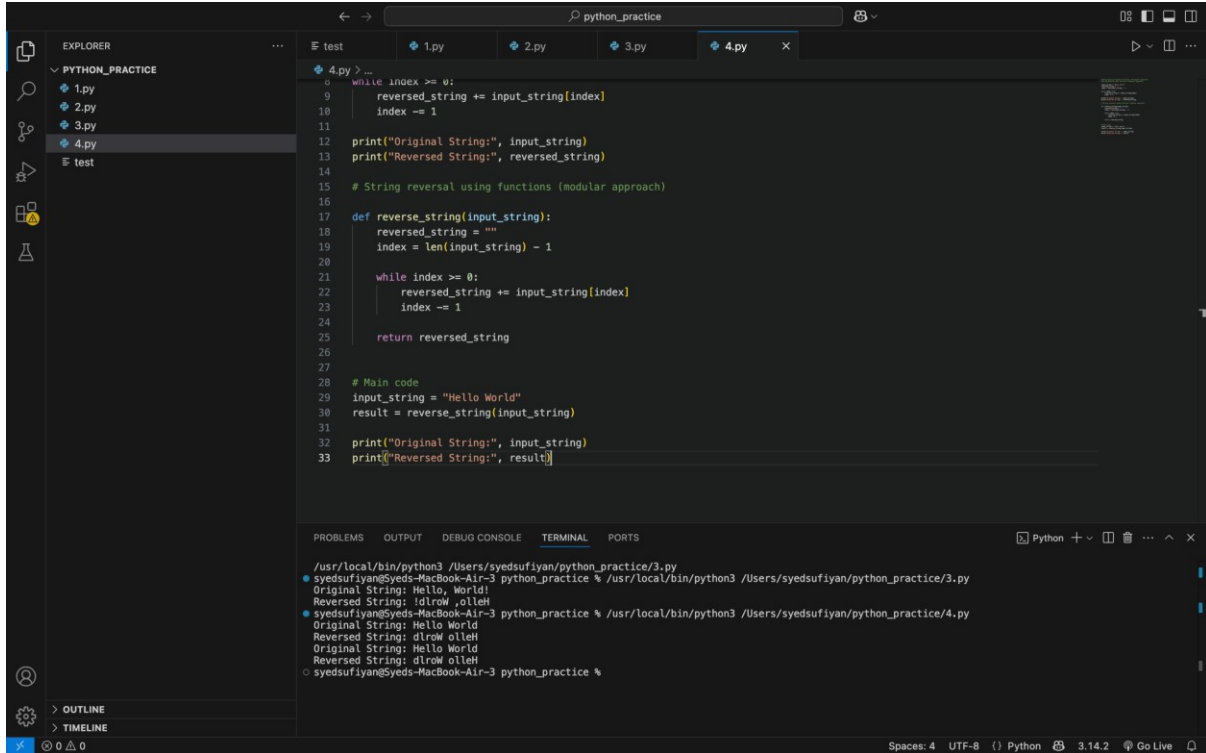
### OBSERVATION:

This program follows a modular design by separating the string reversal logic into a reusable function. The use of clear function names and meaningful comments makes the code easy to understand and maintain. Since the reversal logic is written only once, it can be reused in multiple parts of the application without duplication. Overall, the structure improves readability, reusability, and makes future modifications simple.

### TASK:4

## PROMPT: COMPARATIVE ANALYSIS – PROCEDURAL VS MODULAR APPROACH (WITH VS WITHOUT FUNCTIONS)

CODE:



```
4.py > ...
8 while index >= 0:
9     reversed_string += input_string[index]
10    index -= 1
11
12    print("Original String:", input_string)
13    print("Reversed String:", reversed_string)
14
15    # String reversal using functions (modular approach)
16
17    def reverse_string(input_string):
18        reversed_string = ""
19        index = len(input_string) - 1
20
21        while index >= 0:
22            reversed_string += input_string[index]
23            index -= 1
24
25        return reversed_string
26
27
28    # Main code
29    input_string = "Hello World"
30    result = reverse_string(input_string)
31
32    print("Original String:", input_string)
33    print("Reversed String:", result)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
/usr/local/bin/python3 /Users/syedsufiyan/python_practice/3.py
syedsufiyan@Syeds-MacBook-Air-3 python_practice % /usr/local/bin/python3 /Users/syedsufiyan/python_practice/3.py
Original String: Hello, World!
Reversed String: !dlroW ,olleH
syedsufiyan@Syeds-MacBook-Air-3 python_practice % /usr/local/bin/python3 /Users/syedsufiyan/python_practice/4.py
Original String: Hello World
Reversed String: dlroW olleH
Original String: Hello World
Reversed String: dlroW olleH
syedsufiyan@Syeds-MacBook-Air-3 python_practice %
```

OBSERVATION:

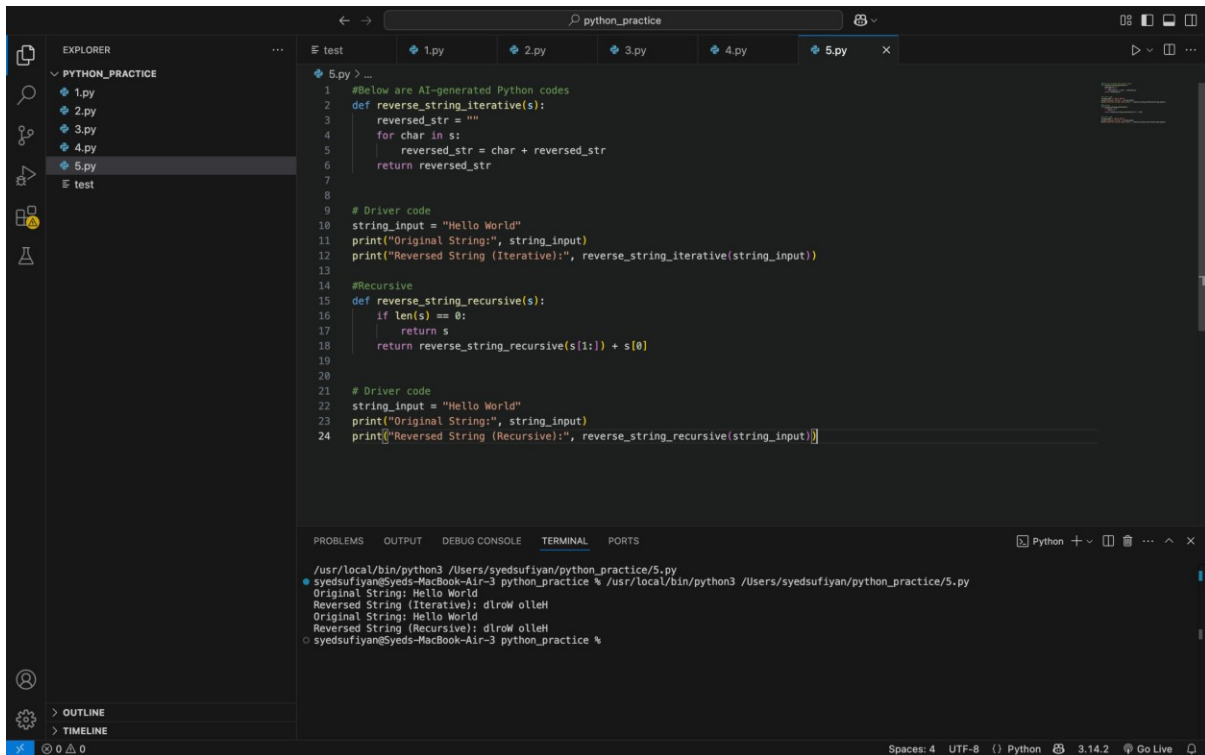
The procedural approach places all logic in one block, making the code harder to reuse and maintain. The modular approach separates logic into a function, improving clarity and structure. Functions allow easy reuse of code without duplication. Debugging is simpler in the modular approach because errors can be isolated. Overall, modular design is better suited for large and scalable applications

TASK:5

PROMPT:

# AI-GENERATED PYTHON CODES ITERATIVE VS RECURSION

## CODE:



The screenshot shows a VS Code editor window with a file explorer on the left and a code editor in the center. The file explorer shows a folder named 'PYTHON\_PRACTICE' containing files '1.py', '2.py', '3.py', '4.py', '5.py', and a 'test' directory. The code editor is open to '5.py' and contains the following Python code:

```
1 #Below are AI-generated Python codes
2 def reverse_string_iterative(s):
3     reversed_str = ""
4     for char in s:
5         reversed_str = char + reversed_str
6     return reversed_str
7
8
9 # Driver code
10 string_input = "Hello World"
11 print("Original String:", string_input)
12 print("Reversed String (Iterative):", reverse_string_iterative(string_input))
13
14 #Recursive
15 def reverse_string_recursive(s):
16     if len(s) == 0:
17         return s
18     return reverse_string_recursive(s[1:]) + s[0]
19
20
21 # Driver code
22 string_input = "Hello World"
23 print("Original String:", string_input)
24 print("Reversed String (Recursive):", reverse_string_recursive(string_input))
```

Below the code editor is a terminal window showing the output of the program:

```
/usr/local/bin/python3 /Users/syedsufiyan/python_practice/5.py
syedsufiyan@syeds-MacBook-Air-3 python_practice % /usr/local/bin/python3 /Users/syedsufiyan/python_practice/5.py
Original String: Hello World
Reversed String (Iterative): dlrow olleH
Original String: Hello World
Reversed String (Recursive): dlrow olleH
syedsufiyan@syeds-MacBook-Air-3 python_practice %
```

## OBSERVATION:

The iterative approach reverses the string by looping through each character, which makes the execution flow easy to follow but slightly slower due to repeated string concatenation. The recursive approach breaks the problem into smaller parts, which is conceptually clean but uses more memory because of function calls and stack usage. Both methods have linear time complexity, but recursion adds extra overhead. For large input strings, the iterative approach is generally safer and more efficient. The recursive method is better suited for

learning and understanding recursion rather than performance-critical applications.